

# Settlement Reconciliation Engine — Razorpay AI Buildathon (Track 04)

---

## The one-sentence pitch

---

Manual settlement reconciliation against Razorpay data typically clears ~51% of rows via spreadsheet VLOOKUP; this engine resolves 90.5% with zero human input, with another 2.7% held for a one-click human confirm (not counted as resolved — see the metrics-bug fix below) — measured on a 514-row batch, ~10x the track's stated 50+ floor, not a single hand-picked example — with every remaining row categorized and explained, not just counted. Matching logic is modeled on how real reconciliation infrastructure works (settlement\_id → bank UTR → order\_id, not amount-first matching), plus one narrowly-scoped, confidence-gated AI layer for the residual cases deterministic matching can't resolve, and a persistent SQLite audit trail so every decision — the pipeline's and a human reviewer's — is a real, inspectable record.

## Architecture, in one diagram

### DATA SOURCES

- Razorpay Settlement Recon API (real, authenticated HTTP call, built and wired -- awaiting your .env secret to actually fire; test mode returns zero items by Razorpay's own design, not a bug here)
- Synthetic bank\_statement.csv, internal\_ledger.csv (no live bank/ERP API exists to plug into for a demo)

v

### INGESTION LAYER (src/ingest.py)

Normalizes real API field names (fee/tax/credit/settlement\_utr) and synthetic CSV names into ONE internal schema before anything touches the matcher. This is where you'd swap synthetic for live data without touching Pass 1-4 at all. Self-tests its own mapping against a hand-verified fixture -- run it.

v

- |           |                                                                                                                                     |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------|
| PASS 1    | settlement <-> bank (key: UTR, tolerance: amount + date)                                                                            |
| PASS 2    | settlement <-> ledger (key: order_id)                                                                                               |
| PASS 2.5  | narration_rules lookup in SQLite (human-confirmed before -- zero fuzzy/LLM needed)                                                  |
| PASS 2.75 | unambiguous exact digit reference (deterministic -- skips the LLM call entirely for cases the arbiter would otherwise rubber-stamp) |
| PASS 3    | fuzzy candidate narrowing (digits + difflib -- only reached when 2.75 found genuine ambiguity)                                      |
| PASS 4    | LLM tie-break, confidence-gated >=0.90 (the ONLY box an LLM ever touches, and only after 1/2/2.5/2.75/3 already failed)             |

v

PERSISTENCE (src/db.py, SQLite, data/reconcile.db)  
Every row's outcome + full replay\_log written to the `exceptions` table. This is what makes "audit trail" a real, inspectable artifact instead of a claim.

v

HUMAN REVIEW SERVER (src/review\_server.py)  
stdlib http.server, localhost:8000, zero dependencies. Lists every OPEN exception. Per row: category, reason, replay\_log (collapsible <details>), and Confirm match / Reject / Needs-clarification. Confirming a FUZZY\_MATCH\_NEEDS\_REVIEW row writes straight into narration\_rules -- no export, no separate promote step.

v

```
REPORTING (src/report.py)  
reconciliation_report.md + exceptions.csv, generated  
from the same results that were persisted to SQLite.
```

Nothing here can move money or finalize a record without either a hard rule (Pass 1/2/2.5), a confidence-gated arbiter that defaults to "ask a human" (Pass 3/4), or an explicit human decision (the review server) — that boundary is the actual point, more than any individual pass.

## Full feature checklist

| #  | Feature                                                                             | Status                                                                                                                                                                                                                                                                                                                                                       | File                                           |
|----|-------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------|
| 1  | Synthetic 3-source data generator, 9 injected failure modes                         | Real MDR (2%), GST (18%), and T+2 settlement cycle, matching Razorpay's current published rates                                                                                                                                                                                                                                                              | <code>generate_data.py</code>                  |
| 2  | Deterministic UTR/amount/date matching (Pass 1)                                     | Built                                                                                                                                                                                                                                                                                                                                                        | <code>reconcile.py</code>                      |
| 3  | Deterministic order_id ledger matching (Pass 2)                                     | Built                                                                                                                                                                                                                                                                                                                                                        | <code>reconcile.py</code>                      |
| 4  | narration_rules lookup, "corrections become reusable rules" (Pass 2.5)              | Built, verified end-to-end over real HTTP                                                                                                                                                                                                                                                                                                                    | <code>db.py</code> + <code>reconcile.py</code> |
| 4b | Unambiguous exact digit reference, no LLM call (Pass 2.75)                          | Built after the arbiter was observed rubber-stamping unambiguous cases — see "Key engineering fixes" below                                                                                                                                                                                                                                                   | <code>reconcile.py</code>                      |
| 5  | Fuzzy narration candidate narrowing (Pass 3)                                        | Built and exercised — an OCR/typo-corrupted order reference is the one case in the dataset that needs it                                                                                                                                                                                                                                                     | <code>reconcile.py</code>                      |
| 6  | Confidence-gated LLM tie-break (Pass 4)                                             | Built and run against a real local model (Ollama, qwen2.5:0.5b) — 4/4 OCR-typo cases correctly picked. Testing also showed the model overconfident on genuinely ambiguous input (picks first-listed candidate, reports $\geq 0.90$ anyway), so a per-tier auto-apply trust gate holds every Ollama result for human review regardless of reported confidence | <code>llm_matcher.py</code>                    |
| 7  | Categorized exception taxonomy (5+3 categories)                                     | Built — <code>ON_HOLD_BY_RAZORPAY</code> reads a real <code>on_hold</code> field already present in ingested data, addressing the most-repeated complaint pattern found in this project's research                                                                                                                                                           | <code>reconcile.py</code>                      |
| 8  | Failure injection / ambiguous-pair demo                                             | Built                                                                                                                                                                                                                                                                                                                                                        | <code>failure_injection_demo.py</code>         |
| 9  | SQLite persistence layer ( <code>exceptions</code> , <code>narration_rules</code> ) | Built                                                                                                                                                                                                                                                                                                                                                        | <code>db.py</code>                             |
| 10 | Human review server (Confirm / Reject / Needs clarification)                        | Built, verified over real HTTP (GET, POST, redirect, DB write). Two-column shell mirrors Razorpay's dashboard structure; colors, shadows, radii, and spacing are taken from Blade's real token files. Includes a real data-                                                                                                                                  | <code>review_server.py</code>                  |

| #  | Feature                                                                                                           | Status                                                                                                                                                                                                                                                                                                                                          | File                                                                                                                    |
|----|-------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
|    |                                                                                                                   | driven donut chart computed from the persisted batch, semantic status pills, and SVG action icons                                                                                                                                                                                                                                               |                                                                                                                         |
| 11 | Report generation (md + csv)                                                                                      | Built                                                                                                                                                                                                                                                                                                                                           | <code>report.py</code>                                                                                                  |
| 12 | Real API field-name mapping (fee/tax/credit → canonical schema)                                                   | Built and self-tested against a verified fixture                                                                                                                                                                                                                                                                                                | <code>ingest.py</code>                                                                                                  |
| 13 | Live Razorpay API integration (test mode)                                                                         | Built, run, and verified against real credentials — authenticated call succeeds, zero items (expected: test mode generates no settlements). Reachable from the actual pipeline via <code>--live</code>                                                                                                                                          | <code>config.py</code> + <code>ingest.py</code> + <code>test_live_connection.py</code> + <code>run_all.py --live</code> |
| 14 | OSS doc PR to <code>razorpay-mcp-server</code>                                                                    | Real branch, real commit, checked against their actual Go source — sitting locally at <code>../razorpay-mcp-server-pr</code> , not yet pushed or opened as a PR, see <code>OSS_PR_DRAFT.md</code>                                                                                                                                               | separate repo                                                                                                           |
| 15 | Load your own bank statement / ledger CSV instead of synthetic                                                    | Built and tested with both a valid and an intentionally malformed file — schema-validated, existing files backed up automatically, clean error messages instead of a stack trace                                                                                                                                                                | <code>load_real_data.py</code> + <code>run_all.py --keep-data</code>                                                    |
| 16 | Gateway-agnostic settlement adapter (proves <code>reconcile.py</code> isn't hardcoded to Razorpay's export shape) | Built and run — 3 orders settling via a differently-shaped, differently-dated export resolved through the exact same Pass 1/2 matching logic, zero changes to <code>reconcile.py</code> . This is strategic reasoning connecting verified facts, not a confirmed Razorpay gap — see "Gateway-agnostic adapter" below before using it in a pitch | <code>generic_gateway_adapter.py</code> + <code>demo_gateway_agnostic.py</code>                                         |

Everything above the "not runnable" line is real, working code, runnable right now with `python run_all.py`.

## What's actually here

```

run_all.py          -- single entry point: runs the batch pipeline
                    (generate, reconcile, inject, persist+report)
                    and prints the full report
src/config.py       -- minimal stdlib .env loader (no python-dotenv
                    dependency) -- reads RAZORPAY_KEY_ID/SECRET
                    from a repo-root .env, never prints them
src/ingest.py       -- normalizes real API field names and synthetic
                    CSV names into one canonical schema before the
                    matcher ever sees a row, AND makes the real
                    authenticated live API call. Run it standalone
                    -- its self-test caught two real bugs during
                    development (`settled` is a bool not a date;
                    the first fix for that then over-corrected and
                    added an unneeded second API call -- both
                    corrected before any live key touched it, see
                    "Key engineering fixes" below).
src/test_live_connection.py -- run once your .env has both credentials:
                    makes one real call, reports the outcome,
                    never prints the key values
src/load_real_data.py -- loads YOUR bank statement / ledger CSV
                    instead of synthetic data. Schema-validated
                    before anything is touched; backs up
                    existing files; clean errors, not a
                    traceback. Use with run_all.py --keep-data.
src/generate_data.py -- synthetic 3-source dataset (settlement/bank/
                    ledger) with 9 deliberately injected real-world
                    failure modes, matching documented Razorpay
                    reconciliation pain points (see Sources below).
                    MDR (2%), GST (18%), and the T+2 settlement
                    cycle are Razorpay's real current published
                    rates.
src/reconcile.py    -- Pass 1/2/2.5/2.75/3/4 matching engine (see diagram)
src/llm_matcher.py -- the ONLY place an LLM plugs in (Pass 4):
                    tries local Ollama first (free, open-weight,
                    the chosen path given real budget
                    constraints), then Anthropic if a key is
                    set, then a deterministic stand-in. Hard
                    confidence gate (0.90) never auto-applies a
                    low-confidence match on any of the three --
                    run it standalone, it names which tier
                    answered in its `reason` field
src/db.py          -- SQLite persistence: `exceptions` (every result
                    row + replay_log) and `narration_rules` (the
                    learned-pattern store Pass 2.5 reads from)
src/review_server.py -- live human-review server, stdlib http.server
                    only (no Flask, no pip install). Confirm /
                    Reject / Needs-clarification per row, writes
                    straight to SQLite -- run it, it opens your
                    browser automatically
src/failure_injection_demo.py -- injects two same-amount, different-UTR
                    transactions and proves the matcher can't
                    cross-wire them, because UTR is the primary
                    key, amount is only a secondary tolerance check

```

```
src/report.py -- persists results to SQLite, then generates
               output/reconciliation_report.md and
               output/exceptions.csv
```

**Quick start:** `python run_all.py` from the repo root runs the whole batch pipeline and prints the full report. Then `python src/review_server.py` opens the live review queue at `http://localhost:8000/` in your browser, backed by the SQLite database the batch run just wrote to.

**Other real modes, not just the default demo:** - `python run_all.py --live` — same pipeline, but the settlement side is a real, authenticated call to Razorpay's API instead of synthetic data (test-mode keys correctly return zero settlements — see "What's honestly not done" below for why that's the right outcome). - `python src/load_real_data.py --bank your_file.csv --ledger your_file.csv` — validates and loads your own bank statement / ledger CSV instead of the generated ones, then `python run_all.py --keep-data` reconciles against them without regenerating or overwriting anything.

Manual run order, if you want to run steps individually (from inside `src/`): `generate_data.py` → `reconcile.py` → `failure_injection_demo.py` → `report.py` → (separately, long-running) `review_server.py`. All paths resolve relative to the repo root via `pathlib`, so this runs unchanged on Windows/macOS/Linux regardless of working directory.

## Key engineering fixes

A running log of the real bugs found and fixed during development, kept because catching your own mistakes before they ship is a stronger signal than pretending the first version was right.

**Correctness** - Duplicate-settlement detection was order-dependent: it only labeled the `_dup` row `DUPLICATE` if random shuffle happened to process it after its non-`_dup` counterpart claimed the bank match. Fixed to compare base settlement IDs symmetrically regardless of processing order. - `llm_matcher.py` and `reconcile.py` were built as two disconnected modules — the confidence gate existed but Pass 3 never called it, so "never auto-applies below 0.90" wasn't exercised by the real pipeline, only by `llm_matcher.py`'s own standalone demo. Fixed: Pass 3/4 now build a real shortlist and hand it to `resolve_with_gate()`. - `ingest.py` went through two corrections before touching live credentials. First: a live recon line's `settled` field is a boolean, not a date — an early version wrongly wired it into `settlement_date`. Second: the fix over-corrected, assuming the recon line carried no date at all and adding a separate batch-level `fetch_all_settlements` call just to get `created_at`. Checking the razorpay-node SDK's own TypeScript definitions showed the recon line has its own `settled_at` timestamp all along — no second call needed. - Day 2's live-API work initially assumed test-mode keys would behave like a smaller version of live data. Razorpay's docs say test mode does not generate settlements at all, so a successful, zero-item authenticated call is the correct outcome, not a bug — `load_settlements(source="live")` and `test_live_connection.py` are built expecting this. - Pass 1's rounding-match fallback had a tie-break bug: when more than one unmatched bank row sat within the  $\pm 2$  tolerance of the same settlement, the loop had no `break` and no comparison, so it silently kept whichever candidate was iterated last, not the closest one. Fixed by tracking the minimum `amt_diff` seen and only replacing the current best on a strictly closer candidate. - A fuzzy-candidate-pool bug: a settlement already resolved to a definite category in Pass 1 (in practice, `DUPLICATE`) remained eligible as a fuzzy-match candidate in Pass 2.75/3/4, because those passes checked `_needs_pass3` but never

whether a category was already set. Confirmed against real OCR-typo narrations that a `DUPLICATE` order's candidate string could score inside diffliB's shortlist cutoff for a narration belonging to a different order — a real risk of corrupting both records at once. Fixed by excluding any record with a category already set from the candidate pool at all three collection points. - `review_server.py`'s confirm endpoint had a genuine concurrency race: `ThreadingHTTPServer` serves concurrent requests, and the confirm/reject write never checked the row was still `OPEN` before overwriting it. Fixed with a conditional `UPDATE ... WHERE resolution_status = 'OPEN'` plus a `cursor.rowcount` check, and a regression test simulating a stale confirm arriving after a real reject. - `_similar_orders()` in `settlement_qa.py` took `next(r for r in rows if r["order_id"] == order_id)`, silently grabbing whichever row was inserted first — but `order_id` isn't always unique (a `DUPLICATE` settlement and its clean-matched sibling share one). Against the real batch this returned the boring `MATCHED` sibling instead of the actual `DUPLICATE` row. Fixed to prefer whichever matching row actually has a category set.

**Reliability / data integrity** - `db.py`'s `persist_results()` deleted prior exception rows with `DELETE FROM exceptions WHERE run_id = ?` before inserting the new batch — but `run_id` is a fresh UTC timestamp on every call, so the delete never matched a previous run's rows, and neither read function filtered by `run_id` either. Net effect: every re-run of the pipeline silently doubled the review queue. Fixed by making `persist_results()` do an unconditional `DELETE FROM exceptions` before inserting, leaving `narration_rules` (the actual cross-run learning store) untouched. - That blanket-delete was later replaced with something that survives a growing daily dataset: upserting on a new `match_key` field instead of deleting and reinserting. A row still `OPEN` gets refreshed by a later run; a row already `CONFIRMED` or `REJECTED` stays frozen, enforced by `ON CONFLICT(match_key) DO UPDATE ... WHERE resolution_status = 'OPEN'` in the SQL itself. A migration path backfills `match_key` for a database created before this existed. - Verifying that upsert surfaced a second bug: `generate_data.py` used `uuid.uuid4()` for `settlement_id / payment_id`, and `uuid4()` draws from `os.urandom`, not Python's seeded `random` module — so despite `random.seed(42)`, every regeneration produced different settlement IDs, meaning `match_key` was never actually stable across a normal re-run. Fixed by generating those hex strings from the seeded `random` module instead, confirmed with an md5 hash of the generated CSVs across two regenerations (byte-for-byte identical). - `ingest.py`'s live Razorpay API call had no retry or timeout handling beyond a bare 15s socket timeout. Fixed: `_live_get()` retries with exponential backoff (1s, 2s, 4s) on network errors and retryable HTTP statuses, up to 3 retries; a 4xx client error fails immediately instead of retrying an auth failure that can't ever succeed. Tested by substituting the transport with a fake that fails twice then succeeds.

**Arbiter (Pass 4) reliability** — running the real Ollama-backed arbiter for the first time surfaced three separate problems: - **Rubber-stamping:** every previously-fuzzy row auto-applied at confidence 1.00, because the model was confirming cases where exactly one candidate's digits already appeared verbatim in the narration — never resolving real ambiguity. Fixed by adding Pass 2.75: an unambiguous exact-digit match now resolves without any LLM call at all. - **Non-determinism:** the same batch, run twice, gave different confidence values for identical rows, from default sampling temperature. Fixed with `temperature: 0`, verified by diffing two runs. - **Overconfidence / positional bias, the most serious of the three:** tested with a narration carrying zero real signal against two equally plausible candidates, the model picked whichever was listed first and reported confidence  $\geq 0.90$  anyway — in one run its own stated reason was "which is the first order in the list." Fixed by adding a per-tier auto-apply trust list:



Ollama results are never auto-applied regardless of reported confidence, always routed to human review.

A second, independent test (`src/ai_judgment_demo.py`) went looking for a case where trusting the model would have been correct, and didn't find one: a ledger narration naming two real orders in one sentence, only one of which it actually says was paid. Three separately framed variants against the real local model: two picked the wrong order outright, and the one that picked correctly still gave an incoherent reason. This is a second, independent reason the trust allowlist stays empty.

**Metrics** - A real metrics-integrity bug, found by tracing the headline number against `db.py`'s own schema: `reconcile.py`'s `summarize()`, `review_server.py`'s `compute_cash_clarity()`, and the Overview donut's `resolved_pct` each independently reimplemented "what counts as resolved" instead of reading `db.py`'s own `needs_action` rule, and all three quietly folded an unconfirmed `MATCHED_LOW_CONFIDENCE` arbiter candidate into "resolved." A second issue sat next to it: `DUPLICATE` rows were counted as separate at-risk cash, double-counting money that had already landed. Fixed: the headline resolved figure moved from 93.2% to the corrected, defensible **90.5%**, with a new third bucket (`cash_pending_review`) surfaced honestly instead of folded into either "resolved" or "still open." Five new regression tests pin this down. - Throughput never appeared anywhere as a headline number, despite the track's bar naming it first ("Throughput plus measured accuracy plus an honest exception list"). Added as its own labeled line in README's Metrics section: **89.8 rows/sec** (514 rows in 5.72s). - The committed 90.5% figure had only ever been validated against the one seed=42 batch it was tuned against. Re-measured against five fresh seeds never tuned against — 7, 21, 99, 555, plus the default 42 — each generating a genuinely different 514-row batch from the same failure-mode mix: **87.1%-90.9%**, every one clear of the ~51% manual baseline by more than 35 points. A `RECON_SEED` environment variable and `extras/seed_sweep.py` make this one command instead of a comment: it runs the real pipeline once per seed, reads the real "Overall resolved" line out of the real generated report, and restores `data/ / output/` to their committed state when done.

**Performance** — two separate bottlenecks, each found by profiling with `cProfile` rather than guessing: - A synthetic 3,000-row stress test found the largest cost was `db.get_narration_rule` opening a fresh SQLite connection and re-running the full schema/migration script once per unmatched ledger row — bigger than the matching logic itself. Fixed with a bulk `get_all_narration_rules()`. Two more linear scans (Pass 2's ledger match, and the Pass 2.5/2.75/3-4 result lookups) were rebuilt as dict indexes built once per run. Result: 3,000 rows went from 6.6s to 0.9s (~7.4x), 500 rows from 0.51s to 0.06s (~8.3x), holding at 3.3s for 6,000 rows. Zero behavior change: all tests passed unchanged, and the real 514-row batch produced byte-identical categorization before and after. - Profiling the actual documented Setup command (`python run_all.py`) on the real batch, rather than a synthetic stress test, showed 32.7 of 41.9 total seconds — 78% of the entire run — spent inside `socket.connect()` for the Ollama arbiter calls, not model inference. Traced to `llm_matcher.py` addressing Ollama by the hostname `localhost` instead of the literal address `127.0.0.1`: on Windows, resolving `localhost` tries IPv6 first, times out, then falls back to IPv4, measured at ~2.05s per connection versus ~0.0004s for the literal address — roughly 5,000x on the connection step alone. Fixed by changing one constant. Measured on the real 514-row batch: **41.88s down to 5.72s, about 7.3x**, with the identical 90.5% resolved figure and identical category counts before and after. The full test suite dropped from roughly 65s to under 11s as a side effect, since several tests exercise the same Ollama-availability check.

**Architecture / observability** - Razorpay's own published Agent Studio guardrails state "every agent action passes through a platform-level validation layer before execution." Until this pass, the confidence gate lived inside `llm_matcher.py`, the same file as the model-calling code it's meant to constrain — a real structural weakness, since nothing prevented a future edit to that file from changing the gate and the matcher together. Fixed by splitting it into `validation_gate.py`, the only file in `src/` that imports `llm_matcher` at all, confirmed with `grep -rln "import llm_matcher" src/*.py` and a structural test that fails if `reconcile.py` ever gains a direct import path to the raw arbiter again. - `replay_log` entries were plain strings. Fixed: every stage entry is now a structured object (`pass`, `action`, `detail`, `confidence`, `timestamp`, `correlation_id`), matching the structured/correlated logging pattern Razorpay's own referenced observability case study describes. `review_server.py`'s renderer handles both the new structured format and a legacy plain-string entry without crashing. - `agent_manifest.json` — a structured, machine-readable governance contract (exact data access, every action allowed/disallowed, the confidence gate rule, idempotency, the learned-pattern specificity guard, revocability) — existed but was never referenced from `README.md`, `PITCH_NOTES.md`, or the deck. Linked from `README`'s "AI Usage and Validation" section, with a one-line comparison to the Razorpay Agent Studio guardrails post it mirrors. - Added `.github/workflows/test.yml`: runs the full test suite plus `run_all.py` on push, across Python 3.10 and 3.12 (3.10 is the real floor, since the codebase uses PEP 604 `str | None` union syntax without `from __future__ import annotations`). Confirmed it passes without Ollama installed: every code path that calls the arbiter falls back to the deterministic stand-in tier on a connection failure. - `report.py` was the one module with zero test coverage — the exact file that turns a computed summary dict into the markdown text a judge reads, the same class of file the metrics bug lived in. Built `tests/test_report.py`: 10 new tests covering every percentage line, the exceptions-by-category table, the cash-position paragraph, the arbiter-invoked phrasing branch, and `exceptions.csv`'s own independent computation. Confirmed the tests have teeth by hand-corrupting a known-good value, confirming the test fails, then reverting.

**Current test count: 122**, re-check with `python -m unittest discover -s tests -p "test_*.py"` before quoting a number out loud, since this file is not re-synced automatically.

## Why this, not a bigger AI system

---

Razorpay already ships production agents across Agent Studio (Dispute Responder, Subscription Recovery, Abandoned Cart, Cashflow Forecaster, RTO Shield/Insights, Settlement Insights) and, more recently, a **Bookkeeping Agent** under Agentic Business Banking that posts accounting entries to a merchant's ERP and explicitly claims "no manual reconciliation required." Read that phrase precisely: **"based on predefined rules."** A rules engine posts entries it recognizes a known pattern for. It cannot, by definition, resolve the case this build exists for — an exception that doesn't match any predefined rule: a narration worded differently across systems, a settlement genuinely ambiguous between two candidates, a GST variance that needs a plain-English explanation before a human can act. Razorpay hasn't published an exception rate for the Bookkeeping Agent — this system's report is that missing number, narrowly and honestly, for the residual layer predefined rules can't and shouldn't auto-resolve. Their own **Analyst, Financial Operations** job listing corroborates that this residual layer is still human-staffed internally.

**This is a systemic-value claim, not just a standalone-gap claim.** Cashflow Forecaster predicts a merchant's cash position 3–7 days out — and a forecast is only as good as the ledger it's built on. If the same unresolved exceptions (duplicate settlements, unexplained bank credits, AFA-blocked charges) sit in a merchant's books uncorrected, Cashflow Forecaster is forecasting on top of noise it has no way to see or fix; that's structural to every forecasting tool that exists — input quality bounds output quality. What's Razorpay-specific is the fix: this build's output is a cleaner, exception-resolved ledger sitting upstream of Cashflow Forecaster's input — "the thing I built makes a product you already shipped more accurate, for free, without either of us touching the other's code."

**This claim ships with a number, not just an argument.** `summarize()` in `reconcile.py` (and the Overview page's "Cash-position clarity" panel in `review_server.py`) computes, from this run's own persisted `net_amount / category / status` columns — no parallel forecast, no invented figure — exactly how much of the batch's rupee value touched some exception/variance path, how much of that this engine resolved deterministically or via a gated match, and how much is honestly still open. A real run: **Rs.48,835 at risk, Rs.20,088 (41.1%) resolved, Rs.28,747 still open** — see `output/reconciliation_report.md`'s "Cash-position clarity" section and `TestSummarize / TestComputeCashClarity` for the tests behind it.

**Found in research, deliberately not chased:** two further gaps surfaced while researching this build — Thirdwatch (a third internal Razorpay system, confirming the same "strong detection, thin explanation" pattern as Bumblebee and Dispute Responder) and a real, undiscussed fairness gap in RTO Shield (pincode-based COD blocking has no visible appeal path for an honest customer in a flagged pincode). Both are genuine, sourced findings. Both are Track 01/02 territory — a customer-facing risk decision, not a back-office finance-ops loop — and neither gets built here.

**The prepared answer to "why didn't you just improve one of the 8 shipped agents":** MediaNama's March 2026 interview with Razorpay's CPO, Khilan Haria, put Dispute Responder's hallucination risk on the record — an agent "geared toward maximising dispute win rates" creating false or misrepresented evidence is a named, public, unresolved concern. This system has **zero LLM-authored financial claims:** matching is deterministic-first (Pass 1/2/2.5), and the one place an LLM touches anything (Pass 4) is scoped to picking a candidate off a shortlist someone else already narrowed, gated so it can never silently act below 0.90 confidence.

**Who this is actually for:** the "user" here is a merchant's finance/ops person, not a shopper — Track 04 has no customer-facing surface at all, and that's a deliberate choice (a customer-facing angle would need its own data model, its own failure-injection story, and its own metrics — a different build). Every design decision reflects that: exception reasons are written in plain accounting language a non-engineer can act on ("GST-on-MDR in ledger differs from settlement report by ₹X — check against your monthly tax invoice," not `tax_diff > threshold`), the review server's draft/pending state exists so someone who isn't going to read source code can add context instead of getting stuck, and the whole interface is one table and three buttons, not a chat window.

**One category deliberately undersold, on purpose:** `AFA_MANDATE_HOLD` names a real, dated regulatory failure mode (the RBI's ₹15,000 e-mandate step-up threshold breaking blind retries) — but Sprint 2026 shows four separate launches visibly hardening exactly this surface (Intelligent Retry Engine, ₹1 UPI Autopay registrations, Mandate Cancellation APIs, card auto-debit limits raised to ₹1 lakh). The claim

isn't "Razorpay hasn't fixed this" — it's "this system correctly recognizes and categorizes a failure mode your own roadmap shows you're actively working on."

## The three upgrades this build adds on top of the original scaffold

Each one maps to a specific finding in this repo's own research docs ( `razorpay-full-roadmap-and-architecture-pattern.md` , `razorpay-failure-log-and-root-pattern.md` ) — none of this is a generic "AI feature," each closes a gap the research explicitly named as open:

1. **The confidence-gated arbiter is wired into the real pipeline**, not just demonstrated in isolation. `reconcile.py` 's Pass 3/4 report three distinct outcomes — `MATCHED_AI_ASSISTED` (arbiter confidence cleared 0.90 on its own), `MATCHED_LOW_CONFIDENCE` (arbiter proposed a candidate, gate held it for a human), and `MATCHED_LEARNED_PATTERN` (Pass 2.5) — instead of collapsing all AI-touched rows into one bucket.
2. **The review surface has a draft/pending state, not just binary approve/reject**. Mirrors the fix pattern from Razorpay's own engineering post, "Breaking the Risk Review Black Box" (Feb 2026): don't force a binary decision when a reviewer doesn't have enough context yet — let them attach a note and come back to it. A row marked "Needs clarification" stays open with the note visible, not dead-ended into a permanent `UNEXPLAINED` .
3. **Human-confirmed matches become reusable rules, not one-off fixes** — the pattern named in Razorpay's own Bumblebee and AI-Security-Triage engineering posts. Confirm a `FUZZY_MATCH_NEEDS_REVIEW` row in the live review server and that exact narration resolves deterministically in Pass 2.5 on every future run — verified end-to-end over real HTTP: a row that needed the arbiter (confidence 0.72, held for review) resolved via `MATCHED_LEARNED_PATTERN` on the very next run after confirming. **Scope, stated honestly:** this memoizes exact narration strings, not a generalized regex/template extraction — the right scope for a repeated/delayed re-export of the same window. Templated extraction is a real next step, intentionally not built.

## Exception taxonomy (matches industry standard, not invented)

`FEE_DEDUCTION` , `TAX_DEDUCTION` , `ROUNDING` , `PARTIAL_PAYMENT` , `UNEXPLAINED` — this taxonomy is used by production reconciliation infra vendors working with real Razorpay settlement data; it's not something built for the demo. Two extra categories go beyond that baseline:

- `AFA_MANDATE_HOLD` flags subscription charges blocked by RBI's April 2026 e-mandate AFA threshold (₹15,000) needing a compliant step-up re-authentication rather than a blind retry — a live, dated regulatory gap.
- `ON_HOLD_BY_RAZORPAY` reads `on_hold` — a real, documented boolean field on the exact settlement recon endpoint this build already calls, confirmed against the official docs and two independent SDKs (`razorpay-java`, `razorpay-php`). This is the cheapest, most directly-evidenced addition in the entire build: one field already present in data already being ingested, mapped to one new category. It exists because "funds held, no reason given" is the single most independently-repeated complaint found across this project's research — Trustpilot, PissedConsumer, G2, and SoftwareSuggest, spanning nearly two years. Without this category, a held payment is indistinguishable from a normal

T+2 delay or a genuinely lost transaction. With it, the exception reason states plainly: known, held, not lost, not a reconciliation error.

**A deliberate deviation from the earliest version of this plan:** an early research pass specced exception categorization as an LLM job ("classify why, generate a one-line explanation"). The actual build categorizes deterministically instead — a GST variance over threshold is `TAX_DEDUCTION` because a number crossed a line, not because a model said so. Given the MediaNama-sourced hallucination-risk concern on Dispute Responder specifically, this is a strictly stronger position: every category label and reason string in this system is generated by a rule a reviewer can read in `reconcile.py`, not a model's judgment call. The one LLM call in the whole pipeline (Pass 4) never writes a category or a reason — it only proposes which `order_id` a shortlist candidate is, gated at 0.90 confidence.

**`TAX_DEDUCTION` is a general GST-on-MDR variance category, not a dedicated TCS/TDS line matcher — a deliberate scoping decision.** Whether this should also handle TCS under CGST Section 52 and TDS under Income Tax Section 194-O was researched from two independent angles: 1. Razorpay's own settlement recon API docs list every field on a recon line item (26 fields: `amount`, `fee`, `tax`, `on_hold`, `settlement utr`, `dispute_id`, and 21 others). There is no TCS, TDS, or Section 52/194-O field anywhere. The only tax field (`tax`) is GST on the processing fee, already correctly read by this category. 2. Section 194-O liability is conditional, not a flag: it falls on whichever entity makes the final payment to the seller, and does not apply if an upstream e-commerce operator already deducted it on the same transaction — real tax-domain judgment, not a threshold a reconciliation engine can read off a settlement line. Section 52 TCS applies to marketplace operators paying third-party sellers, not to the direct-to-consumer merchant scenario this build (and Track 04's own framing) models. 3. Terra Insight, a payment-gateway reconciliation research firm, states directly that "pure direct-to-consumer merchants using Razorpay as a payment gateway do not attract TCS on their Razorpay settlements." Even Razorpay's Route product (linked accounts, split payments to marketplace vendors) has no TCS/TDS field on its transfer/settlement API. RazorpayX's separate "Tax Payments" feature computes the liability from a formula the merchant runs manually — the same approach this build's own `TAX_DEDUCTION` category already takes for GST-on-MDR.

Building this properly would mean either fabricating settlement data Razorpay doesn't produce, or growing a full tax-compliance decision engine alongside the reconciliation one — scope creep this build has avoided elsewhere. Checked a third time against the newest Razorpay material available: Sprint '26 (May 2026) shipped "Automated TDS Payments and Filing" under RazorpayX Business Banking+, but it automates TDS on outgoing vendor payouts a business makes, not incoming settlement reconciliation via the Settlement Recon API — the mirror-image surface of what this build reconciles, not an overlap. Source: [razorpay.com/blog/business-banking/automated-tds-filing-from-razorpayx-business-banking](https://razorpay.com/blog/business-banking/automated-tds-filing-from-razorpayx-business-banking)

## The live Razorpay API call

`python src/test_live_connection.py` made a real authenticated call to `GET /v1/settlements/recon/combined` with test-mode keys: `SUCCESS -- authenticated call completed`. Items returned: 0 — the predicted result, confirmed by actually running it. The full pipeline end to end against it (`python run_all.py --live --keep-data`): 161 rows persisted, 100% `UNEXPLAINED` (0 live settlements means every synthetic bank/ledger row has no counterpart), overall resolved 0.0% — the expected shape of a live test-mode run. The honest claim is "the integration is real, authenticated, and



correctly shaped; the emptiness is Razorpay's own test-mode behavior." (That live-mode run was cleared from `data/reconcile.db` — a local, gitignored file — so the demo review site shows the clean synthetic batch, not this verification run's noise.)

## What's honestly not done

- `llm_matcher.py`'s `call_llm_arbiter` tries three tiers in order: **Ollama** (a real, local, open-weight model — the chosen path, since paid API credit wasn't an option for a solo student build), then **Anthropic** if `ANTHROPIC_API_KEY` is set and Ollama wasn't reachable, then the original deterministic stand-in if neither is reachable. `_call_anthropic` makes a raw HTTPS call to the real Messages API, no SDK, so the zero-third-party-dependency claim stays true. Tested against the real API with a configured key: authentication and request format are both confirmed correct (what came back was a real, structured billing error — "Your credit balance is too low" — not a code defect). `_call_anthropic` returning `None` on that error and `call_llm_arbiter` correctly falling through to the stand-in is the fallback chain working as designed, covered by `TestCallAnthropic` and `TestCallLLMArbiter`'s fallback tests, which substitute the network call rather than asserting against a real response this account can't produce.

**Anthropic is not on `AUTO_APPLY_TRUSTED_TIERS` either, same as Ollama** — not because it's known to be unreliable (the opposite of Ollama's case, which was proven unreliable by a real adversarial test), but because a tier that has never actually returned a live judgment has zero evidence behind it either way. The confidence threshold (`Pass 4`, `resolve_with_gate`) is identical regardless of tier; which tiers are allowed to auto-apply at all is not, and today that answers "none," for two different reasons per tier. The task itself (pick one candidate off an already-narrowed 1-3-item shortlist) doesn't need a frontier model's reasoning, only reliable structured output plus honest self-doubt — Ollama's JSON-schema enforcement gives reliable output for zero cost, but testing showed it can't be trusted for the self-doubt half, so the system compensates in code rather than assuming the model would behave. - `UNEXPLAINED` rows are genuinely unresolved — that's correct behavior, not a bug. A reconciliation tool that claims 100% match rate is the thing to be suspicious of, not proud of. - The learned-pattern store (`narration_rules` in `db.py`) memoizes exact narration strings, not a generalized template/regex. It will not help if next month's messy narration is worded differently for the same underlying customer — that needs templated extraction, a real next step, not built here. - `review_server.py` only writes a `narration_rules` entry on a `FUZZY_MATCH_NEEDS_REVIEW` confirm, by design — confirming a `DUPLICATE` / `UNEXPLAINED` / `AFA_MANDATE_HOLD` row is confirming a settlement-integrity finding, not a narration-parsing fact, so it's deliberately never written into the pattern store. - `review_server.py` is single-user, unauthenticated, localhost-only. Correct for a solo demo, not a production claim — there's no login, no concurrent-reviewer conflict handling, and it should never be exposed past `localhost`.

## What breaks at scale, and what an adversarial input does to it

- **At 10x-100x volume, lookups needed an index, not a scan.** A synthetic 3,000-row batch was built (isolated benchmark script, not part of the test suite) and profiled with `cProfile`. Three things showed up, in order of actual cost: 1. `db.get_narration_rule` was the single largest cost (~2.8s of ~9.7s), bigger than the matching logic itself — it opened a brand new SQLite connection and re-ran

the full schema/migration script on every unmatched ledger row. Fixed with

`db.get_all_narration_rules()`, one query loading every learned rule into a dict up front. 2. Pass 2's ledger match and the Pass 2.5/2.75/3-4 result lookups were exact linear scans, each an  $O(n)$  scan repeated for every settlement or unresolved row. Fixed with two dicts built once (`ledger_by_order_ref`, `results_by_order_id`), mirroring the `bank_by_utr` index Pass 1 already used. 3. Pass 2.75/3's exact-digit-reference check recomputed `oid.split("_")[1]` for the same order on every ledger row (9 million calls on the 3,000-row batch) instead of once per order per pass. Fixed by hoisting the split out of the inner loop. The substring containment check itself is deliberately left  $O(\text{candidates})$  per ledger row — collapsing it further would change what counts as a match for edge cases no test or synthetic row exercises, a correctness risk not worth taking for a further constant-factor win.

Measured result, same 83/83 tests passing unchanged: 3,000 rows went from 6.6s to 0.9s ( $\sim 7.4\times$ ). 500 rows: 0.51s to 0.06s ( $\sim 8.3\times$ ). At 6,000 rows, 3.3s. These figures came from an isolated stress-test script (`bench_scale.py`, not part of this repo) that touches no real database — the actual demo dataset (514 rows) runs in  $\sim 40$ s dominated by Ollama LLM calls for the  $\sim 15$  rows that reach Pass 3/4, not by the matching-logic scans this fix targeted. - **A narration with two valid-looking order references breaks the disambiguation.** Something like "refund for order 1077, original order 1042" contains two 4-digit sequences that both satisfy the exact-substring shortlist rule. Current behavior picks whichever appears first in iteration order — arbitrary. A real fix would need the shortlist to weight proximity to disambiguating keywords ("original," "refund," "for") rather than just extracting digits; not built, named as a real blind spot. - **The learned-pattern store could be poisoned by an overly generic confirm — now guarded.** `db.resolve_exception` used to trust that a confirmed `FUZZY_MATCH_NEEDS_REVIEW` narration uniquely identified one transaction; a boilerplate confirm could have memoized a string that could plausibly describe many different orders. `db.is_narration_specific` now refuses to write a `narration_rules` entry unless the narration contains a digit run that is this order's own numeric suffix and that suffix isn't shared with any other order in the exceptions table. The confirm itself still succeeds either way — only the auto-apply-next-time memoization is withheld when the narration isn't specific enough to trust. - **review\_server.py has no concurrency control** beyond the fix above. Two reviewers hitting Confirm on the same row at nearly the same instant could both succeed at the SQL level (SQLite serializes the writes), but there's no optimistic-lock response telling the second reviewer someone already resolved it. Fine for a solo demo with one reviewer; a real multi-reviewer deployment would need that response. - **Three real Ollama reliability issues (non-determinism, rubber-stamping, overconfidence/positional bias)** are detailed under "Key engineering fixes" above. The residual risk even after the per-tier auto-apply trust gate: a human reviewer can still be persuaded by a fluent, confident-sounding `reason` string from a model already proven to guess with unwarranted confidence. The gate stops the system from trusting Ollama's confidence; it can't stop a reviewer from trusting its prose without independently checking the narration. That dependency on reviewer diligence is real and not solved by software here.

## Real API schema note (verified against Razorpay's own docs/SDKs)

The synthetic data in this repo uses simplified column names (`gross`, `mdr`, `gst_on_mdr`, `net`) for readability. The actual Razorpay Settlements API is two-tiered and shaped differently — now coded in `ingest.py`, not just described here:

- A `settlement` entity is a **batch**: `id` (`setl_...`), `amount`, `fees`, `tax`, `utr`, `created_at`. No `order_id` here — not the row this build actually needs for matching.
- Per-order detail comes from the **Settlement Recon** endpoint, `GET /v1/settlements/recon/combined?year=&month=[&day=]` (the MCP server's `fetch_settlement_recon_details` tool): line items shaped as `entity_id` (`pay_...` / `rfnd_...`), `type` (`payment` / `refund`), `debit`, `credit`, `amount`, `fee`, `tax`, `settled` (a boolean, "has this line settled" — not a date), `created_at` (when the underlying payment was made — also not the settlement date), `settled_at` (a unix timestamp — the actual per-line settlement date), `settlement_id`, `settlement utr`, `order_id`, `method`, `card network/issuer`, `dispute_id`.
- There is no single `gst_on_mdr` field in the real API — it's `fee` (the MDR) and `tax` (GST charged on that fee) as two separate numbers.
- **The recon line carries its own date ( `settled_at` ) — no second, batch-level call is needed just to reconcile.** `fetch_all_settlements` is still useful for batch-level totals, but `load_settlements(source="live")` only needs the recon endpoint to build every canonical row this system matches on.
- The two-tier UTR (batch-level `utr` vs. line-level `settlement utr`) is itself a real, underappreciated source of merchant confusion — now a built exception category, `UTR_LEVEL_MISMATCH`, distinct from the 5 in the industry-standard taxonomy above. `reconcile.py`'s Pass 1 checks every unmatched bank row (not just candidates under this settlement's own UTR) for one matching amount+date exactly, and resolves it as a UTR mismatch — money arrived, wrong reference — only if exactly one such row exists; two or more is a genuine coincidence and is left unresolved. The scan excludes any bank row whose UTR is claimed as some other settlement's own reference, so it can never steal a row that legitimately belongs to a different, not-yet-processed settlement. See `TestUTRLevelMismatch` in `test_reconcile.py`.

`ingest.py`'s mapping (`gross` → `amount`, `mdr` → `fee`, `gst_on_mdr` → `tax`, `net` → `credit`, `utr` → `settlement utr`, `settlement_date` → `settled_at`) is coded and self-tested — run `python src/ingest.py` and read its own output.

## The gateway-agnostic adapter — why it exists, and the exact limits of the claim

**Read the framing carefully before using this in a pitch.** Everything else in this document that cites a fact (the 51%/88% benchmark, the Bookkeeping Agent's "predefined rules" phrasing, the MDR/GST/T+2 rates) is something Razorpay or a credible third party stated directly. This section is different: it's strategic reasoning, built by connecting several independently verified facts — not something Razorpay has confirmed wanting. Presented as a hypothesis to defend, not a fact to cite.



**The reasoning, each piece checked independently:** - Merchants running multiple payment gateways in parallel is mainstream — Juspay and GoKwik's Kwik Checkout are orchestration products that exist specifically to route one merchant's traffic across Razorpay, Cashfree, and PayU simultaneously. - Razorpay's own CEO, Harshil Mathur, on record: "the idea is to evolve Razorpay beyond a payments provider into a financial operating system for businesses" (BusinessToday, March 2026). - COO Rahul Kothari, same period: "we're rebuilding our entire financial operating system to be AI-first." - Finnato — a separately funded company — exists specifically to sell "a single source of financial truth" across a merchant's tools, real market proof this is a validated problem category. - A reconciliation tool that only understands Razorpay's own settlement data is, structurally, still a payments-provider tool wearing an "operating system" label — an inference connecting the above, not a Razorpay statement.

**What was actually built to back the reasoning, modestly:** `reconcile.py` 's Pass 1-4 matching logic resolves a settlement from a second, completely-differently-shaped export (`generic_gateway_adapter.py` 's "Gateway B" — different column names, different date format) with zero changes to the matching code. `demo_gateway_agnostic.py` injects 3 orders settling via this second shape and confirms all 3 resolve through the same Pass 1/Pass 2 logic as Razorpay-sourced settlements.

**What this deliberately does NOT claim:** it does not claim "Gateway B" matches any real competitor's actual API or file format — it's a generic stand-in for gateway heterogeneity in general. It does not claim Razorpay has asked for this, confirmed a gap here, or would want it built this way.

**How to run it** (separate from the main pipeline on purpose — a "what's next" capability, not folded into `run_all.py` 's default steps):

```
python src/generate_data.py          # fresh baseline
python src/demo_gateway_agnostic.py  # injects 3 Gateway-B orders, proves the match
```

Like `failure_injection_demo.py`, this appends rows to `bank_statement.csv` and `internal_ledger.csv` — run `generate_data.py` again afterward to reset to a clean baseline.

## `extras/smart_collect_reconcile.py` — **proof the gate generalizes, not a second product**

**Deliberately standalone**, not wired into `run_all.py` or the review UI. Settlement reconciliation (`reconcile.py`) is one-to-one: one settlement row maps to one order. Razorpay's real Smart Collect product (virtual accounts) is many-to-one: many different payers send money into one shared account, each payment needing attribution from whatever weak signal exists. This script proves the confidence gate holds up on that structurally different shape, without rebuilding it.

**What's actually reused, not reimplemented:** `validation_gate.py` 's `resolve_with_gate` is imported directly — same 0.90 threshold, same trusted-tier rule, the same function `reconcile.py` itself is restricted to calling.

**Run it and read real output** (seed=7, fully reproducible):

```
python extras/smart_collect_reconcile.py
```

Actual output from this run: 22 credits, 45.5% clean match, 4.5% matched with variance (over/underpayment), 45.5% correctly held for human review, 4.5% genuinely unattributable exception. The trap case — two different payers sending the identical amount with no other signal — correctly lands as `MATCHED_LOW_CONFIDENCE / EXCEPTION`, never a guess.

**What this surfaced, unprompted:** with only four common fee amounts across 20 dues, amount-only candidate lists get long fast (one credit in the sample run had 10 equally-plausible candidates on nothing but a shared amount) — far more collision-prone than settlement UTRs ever are, since a UTR is unique per bank credit by construction. That's not a bug in this proof script; it's the honest finding that virtual-account attribution is a genuinely harder version of the same problem, not just a smaller one.

## Adoption path — who actually uses this, and how it would actually ship

---

Track 04 has no customer-facing surface by design, so "GTM" here means the same question Razorpay's own past hackathon winners actually had to answer: who inside or around Razorpay would pick this up, and why. Checked against every Razorpay hackathon winner independently verifiable in this research (DrishtiPay — RBI HaRBInger 2023, 1st prize, now shipped as Divyang Drishti Pay on POS; Razorpay OAuth — Status 402 hackathon, now an actual product feature; Discount Whisperer — internal hackathon, "concept to production-ready in 72 hours"), every single one shares one trait: it was real enough to become an actual shipped feature or internal tool, not a demo that stayed a demo.

**Primary user:** a merchant's finance/ops person (or the "Analyst, Financial Operations" role Razorpay's own careers page confirms still does this by hand) — not a shopper, not a merchant founder. Already reflected in the exception language throughout the system (plain accounting terms, not engineer-speak).

**Adoption path inside Razorpay, concretely:** the natural home for this isn't a new standalone product — it's the exception-resolution layer underneath the existing Bookkeeping Agent (Agentic Business Banking). The Bookkeeping Agent already posts "based on predefined rules"; this system is the thing a merchant's finance team would reach for the moment the Bookkeeping Agent hits a case its rules don't cover, which per its own marketing language is guaranteed to happen. `narration_rules` (Pass 2.5) and the SQLite audit trail are already structured the way a "human-in-the-loop exception queue" for an existing agent product would need to be.

**Target segment, if this shipped as a standalone add-on instead:** D2C merchants doing high settlement volume with multiple payment methods (UPI + cards + refunds), still reconciling by spreadsheet — exactly the Financial-Ops-Analyst-shaped gap the careers listing confirms exists today. The pitch to that merchant is one sentence: *87% resolved automatically against roughly 51% for manual VLOOKUP, with every remaining exception explained instead of just counted* — a number a finance lead can act on immediately.

**What this deliberately doesn't claim:** a pricing model, a sales motion, or a rollout plan — none of that is this track's job. The honest GTM claim for a solo buildathon submission is narrower: this is the shape of thing that becomes a real feature at this company, evidenced by every past winner that did exactly that.

## Sources

---

- `on_hold` field, verified three ways: the official docs page ([razorpay.com/docs/api/settlements/fetch-recon](https://razorpay.com/docs/api/settlements/fetch-recon)), and independently confirmed identical in both the razorpay-java and razorpay-php SDK documentation ([github.com/razorpay/razorpay-java](https://github.com/razorpay/razorpay-java), [github.com/razorpay/razorpay-php](https://github.com/razorpay/razorpay-php), `settlement.md` in each). "Funds held, no reason given" as the most-repeated complaint pattern: Trustpilot (multiple reviews, Sept 2024-June 2026), PissedConsumer, G2, SoftwareSuggest.
- Gateway-agnostic reasoning: Harshil Mathur's "financial operating system" quote (BusinessToday, March 2026); Rahul Kothari's "AI-first" quote; Finnato's "single source of financial truth" positioning ([finnato.com/vision](https://finnato.com/vision)); Juspay and GoKwik's multi-gateway orchestration products as evidence multi-gateway merchants are mainstream. Presented as inference, not a confirmed Razorpay gap — see that section above.
- Razorpay hackathon winner pattern (adoption-path evidence above): DrishtiPay / RBI HaRBInger 2023 ([razorpay.com/newsroom](https://razorpay.com/newsroom), [business-standard.com](https://business-standard.com)), Razorpay OAuth / "Status 402" hackathon ([razorpay.com/blog](https://razorpay.com/blog)), "Discount Whisperer" ([engineering.razorpay.com](https://engineering.razorpay.com)), HACK:O(n) 2024 scale ([yourstory.com](https://yourstory.com)). A claimed "Hack:(0)n 2025" winner list including team names could not be independently verified through any public source and is not relied on above.
- Ollama's JSON-schema-enforced structured outputs (used to constrain Pass 4's response format when running locally): [docs.ollama.com/capabilities/structured-outputs](https://docs.ollama.com/capabilities/structured-outputs)
- 51% manual / 88%+ structured match-rate benchmark, `settlement_id`-as-match-key (not UTR), and the 5-category exception taxonomy: Terra Insight, "Razorpay Settlement Reconciliation: Unpacking Net Payouts" ([terra-insight.com](https://terra-insight.com))
- Razorpay Agent Studio's shipped agent list and guardrail philosophy (merchant control, review-first mode, platform validation layer, audit trail): [razorpay.com/blog/razorpay-agent-studio-principles-guardrails-and-merchant-control](https://razorpay.com/blog/razorpay-agent-studio-principles-guardrails-and-merchant-control)
- Razorpay settlement report schema (`settlement_id`, UTR, MDR, GST on MDR, net amount fields): [razorpay.com/docs/api/settlements/entity](https://razorpay.com/docs/api/settlements/entity)
- Settlement Recon endpoint schema (`entity_id`, `settlement_utr`, `order_id`, `fee`, `tax`, `settled_at`): [razorpay.com/docs/api/settlements/fetch-recon](https://razorpay.com/docs/api/settlements/fetch-recon)
- Independent cross-check of the recon line's `settled_at` field: razorpay-node SDK's own TypeScript definitions, `settlements.d.ts` ([unpkg.com/razorpay](https://unpkg.com/razorpay))
- Razorpay MCP Server tool catalog (`fetch_all_settlements`, `fetch_settlement_with_id`, `fetch_settlement_recon_details`): [github.com/razorpay/razorpay-mcp-server](https://github.com/razorpay/razorpay-mcp-server)
- Financial Ops Analyst role (confirms reconciliation is still human-staffed internally): Razorpay careers listing, "Analyst, Financial Operations"
- Bookkeeping Agent / Receivables Agent, Agentic Business Banking: [razorpay.com/blog/business-banking/introducing-the-agentic-business-banking-experience-by-razorpayx](https://razorpay.com/blog/business-banking/introducing-the-agentic-business-banking-experience-by-razorpayx)
- "Breaking the Risk Review Black Box" (Feb 2026), Bumblebee (Dec 2025), and "From 750 Hours to 2 Hours" AI Security Triage (Jun 2026): Razorpay Engineering blog ([engineering.razorpay.com](https://engineering.razorpay.com))
- RBI Digital Payments – E-mandate Framework, 2026 (₹15,000 AFA threshold, effective April 21 2026): RBI circular, corroborated by Business Standard, Conventus Law, Agrud Partners coverage

- Real documented mandate-retry failure at the ₹15,000 threshold: Shopify Community forum post, "Invoice Payment Issue for Indian Merchants Due to RBI Mandate Change"
- Intelligent Reconciliation Agent (two-source, screenshot-driven UTR/amount match, "flag a discrepancy"), part of the Agentic Dashboard: [razorpay.com/blog/razorpay-agentic-platform/](https://razorpay.com/blog/razorpay-agentic-platform/)
- Track 04's own bar text, the Sept 5 2026 deadline, and confirmation that "audit trail and one failure handled gracefully" is Track 01's bar, not Track 04's: [razorpay.com/buildathon/](https://razorpay.com/buildathon/)
- "Automated TDS Payments and Filing" (Sprint '26, RazorpayX Business Banking+, outgoing vendor payouts, not incoming settlement reconciliation): [razorpay.com/blog/business-banking/automated-tds-filing-from-razorpayx-business-banking](https://razorpay.com/blog/business-banking/automated-tds-filing-from-razorpayx-business-banking)

## Execution status

---

Live API connected and fired against real test-mode credentials. Ollama installed and running the real arbiter end to end. Review server built and verified over real HTTP, including the confirm-to-learned-pattern loop. Scale and adversarial-input limits documented under "What breaks at scale" above. Demo video script and shot list live in `VIDEO_SCRIPT.md`.

## Contact

---

**Repository:** <https://github.com/niyat-ati/recon-engine> **Email:** [adityamukherjee158@gmail.com](mailto:adityamukherjee158@gmail.com)